PROJECT REPORT · COMPUTER VISION · SIGN LANGUAGE AI

ISL RECOGNITION

Indian Sign Language Recognition System
Two-Handed Real-Time AI Pipeline

DOCUMENT ID : SP_ASEP / JUNE 2026

PART 01	Problem Statement, Background & Full System Overview	LARGEST	MEMBER 1
PART 02	Frontend — UI, Visualization, TTS & All Features	EQUAL	MEMBER 2
PART 03	Backend, Socket.IO, MediaPipe & ML Pipeline	EQUAL	MEMBER 3
PART 04	Dataset Collection, Training & Deployment Guide	SHORTEST	MEMBER 4

PART 01 · MEMBER 1 · LARGEST SECTION

Problem Statement & Full System Overview

Background, motivation, objectives, how the system works end-to-end, and why each technology was chosen

◆ PROBLEM STATEMENT & OVERVIEW

1.1 Background — Who Needs This?

India has approximately **6.3 million people** with hearing and speech impairments — one of the largest deaf communities in the world. Their primary language is **Indian Sign Language (ISL)**, a rich visual language with its own grammar, vocabulary, and sentence structure, completely independent of any spoken Indian language.

The problem: almost none of the hearing population can understand ISL. This creates a daily barrier in hospitals, schools, banks, job interviews, and basic social interaction. Professional interpreters are rare, expensive, and unavailable in most situations.

> **IN SIMPLE TERMS — HOW TO EXPLAIN THIS**
>
> Imagine you can only communicate by waving your hands in specific patterns, but everyone around you only understands spoken language — and there's almost never a translator nearby. That's the daily reality for millions of hearing-impaired people in India. Our system is like a live translator that watches the hand signs through a camera and instantly converts them into words and spoken audio — no special hardware, no interpreter, just a laptop and a webcam.

1.2 Problem Statement

Existing ISL translation tools fail on one or more of these four dimensions:

- **Hardware dependency:** Most research systems require expensive depth cameras (e.g. Microsoft Kinect, ~₹25,000) or sensor gloves — unaffordable for most users.
- **Limited vocabulary:** Previous datasets only cover one-handed alphabet signs, ignoring the two-handed gestures that make up a large portion of real ISL.
- **No real-time output:** Batch-processing systems analyze recorded video after the fact — they cannot support live conversation.
- **Unusable interfaces:** Research prototypes produce raw code output, not a usable product for real users.

> **◆ OUR SOLUTION**
>
> A real-time, browser-based ISL recognition system that uses only a standard webcam, recognizes both hands simultaneously, builds complete sentences, and speaks them aloud — running on any laptop without any special hardware.

1.3 Project Objectives

1. Recognize all **26 ISL alphabet signs (A–Z)** and **9 full-word gestures** (HELLO, LOVE, THANKYOU, HELP, SORRY, YES, NO, GOOD, PLEASE) in real time.
2. Support **two-handed signs** by tracking both hands simultaneously and measuring their positions relative to each other.
3. Build a complete **sentence pipeline**: detected letter → word building → word suggestions → sentence → text-to-speech.
4. Include a **built-in trainer** so anyone can record their own sign data and train a new model without writing any code.
5. Run on both **Linux and Windows** with a single command and a standard webcam.

1.4 How the System Works — End to End

The system has four stages that work simultaneously as a pipeline:



1. **Capture:** The webcam captures a video frame. The frame is flipped horizontally so it acts like a mirror.
2. **Detect:** MediaPipe locates the hands in the frame and produces 21 landmark points (fingertip, knuckle, wrist, etc.) for each hand in 3D coordinates.
3. **Extract:** The 21 points from each hand (plus 6 measurements of how the two hands relate to each other) are converted into 132 numbers that represent the sign in a scale- and position-independent way.
4. **Classify:** A trained machine learning model takes those 132 numbers and predicts which ISL sign it is (A, B, HELLO, etc.) along with a confidence percentage.
5. **Stabilize:** The same prediction must appear for 8 consecutive frames before it is accepted — this prevents flickering or accidental detection during hand movement.
6. **Display & Speak:** The detected letter is shown in the browser, added to the current word, and eventually the full sentence is spoken aloud.

> **IN SIMPLE TERMS**
>
> Think of it like this: the webcam is your eyes, MediaPipe is a hand-tracker that finds exactly where each finger and knuckle is, the feature extractor turns those positions into a list of 132 measurements (like a fingerprint of the hand shape), and the Random Forest is the brain that compares that fingerprint against all the hand shapes it learned during training and says "this looks like the letter A with 94% confidence." The 8-frame rule is like a teacher saying "hold the sign steady for a moment before I accept it."

1.5 System Architecture — Three Threads

The system runs three parallel background processes (threads) connected by queues, so that video capture, AI processing, and streaming to the browser all happen simultaneously without blocking each other:

```
Thread 1 — Grabber :  Webcam → flip → raw_queue
Thread 2 — Detector:  raw_queue → MediaPipe → 132 features → Random Forest → encode_queue
Thread 3 — Encoder :  encode_queue → JPEG compress → Socket.IO → Browser canvas
```

If the detector is slow on a particular frame, the grabber keeps capturing new frames and drops the old one — the system never freezes. The browser always shows the most recent frame, never a queue backlog.

1.6 Technology Stack — What We Used and Why

Component	Technology	Why This Choice
Hand Detection	MediaPipe Hand Landmarker	Free, runs on CPU (no GPU needed), detects 2 hands, gives precise 3D landmarks at 30+ fps
ML Classifier	Random Forest (scikit-learn)	Trains in seconds on small datasets, high accuracy with as few as 100 samples per class, no GPU required
Server Framework	Flask + Flask-SocketIO	Lightweight Python web server; Socket.IO enables real-time two-way communication for live video streaming
Video Capture	OpenCV (cv2)	Industry standard for camera access and image processing across Linux and Windows
Frontend	HTML + CSS + Vanilla JS	No framework needed; runs in any browser, no install required for the user
Text-to-Speech	Browser Web Speech API	Built into Chrome/Edge, works on any OS, uses the system's own voices without any extra packages
Language	Python 3.10+	Best ecosystem for AI/ML (NumPy, scikit-learn, MediaPipe all have Python APIs)

1.7 Why Two Hands Matter for ISL

Many ISL signs look identical if you only watch one hand. For example, the difference between letters **M** and **N** in ISL is in how many fingers of one hand touch the other hand's palm. A system that only tracks one hand cannot distinguish these signs.

Our model captures **63 measurements from the right hand + 63 from the left hand + 6 measurements of how the two hands relate to each other = 132 total**. This is a significant improvement over previous single-hand ISL systems which used only 63 features and could not handle two-handed signs at all.

◆ KEY INNOVATION

The 6 "relative features" — the X, Y, Z distance between wrists, the overall wrist-to-wrist distance, the index fingertip gap, and the thumb tip gap — allow the model to understand two-handed signs as gestures involving *both hands together*, not just two independent hand shapes.

1.8 ISL Signs Covered

Alphabets (A–Z)

All 26 letters of the ISL fingerspelling alphabet. Users spell out words they don't have a specific gesture for, letter by letter. The system builds the word and suggests completions automatically.

Full Word Gestures (9)

HELLO, LOVE, THANKYOU, HELP, SORRY, YES, NO, GOOD, PLEASE — these common words have dedicated two-handed gestures recognized as complete words instantly, without letter-by-letter spelling.

PART 02 · MEMBER 2 · EQUAL WEIGHT

Frontend Development

UI design system, real-time video rendering, recognition display, TTS integration, and all user-facing features

◆ FRONTEND & USER EXPERIENCE

2.1 What the Frontend Does

The frontend is the part of the system the user actually sees and interacts with — the website that opens at `http://localhost:5000`. It is built with plain HTML, CSS, and JavaScript (no external framework like React or Vue). Its jobs are: show the live camera feed with landmarks drawn on it, display the current detected sign and confidence, let the user build sentences, and convert those sentences to speech.

> **IN SIMPLE TERMS**
>
> The frontend is the "face" of the application. Everything the user can see and click — the live camera video, the big glowing letter, the suggestion buttons, the SPEAK button — is part of the frontend. The Python server is invisible to the user; the frontend is what makes the system feel like a real product rather than a research experiment.

2.2 Design System — Colors, Fonts, Layout

A bespoke design system was built with deliberate choices that reflect the subject — sign language as warm, human communication, not cold machine surveillance:

- **Amber (#E8A225)** as the main accent color — warm, human. Replaces the cold blue/cyan typical of AI dashboards.
- **Teal (#00C4A7)** for detection data (hand indicators, stability bar).
- **Deep navy (#07080D)** background — dark enough to make the glowing accents pop.
- **Syne** font for the large letter display — geometric, bold, highly readable at huge sizes.
- **DM Sans** for body text, **JetBrains Mono** for numbers and data.

The layout is a fixed **three-column grid**: live camera feed on the left, recognition logic in the center, ISL alphabet reference on the right — all visible simultaneously so users never need to switch tabs during signing.

2.3 Real-Time Camera Feed (Canvas + WebSocket)

The camera feed is NOT a standard `<video>` element. Instead, the server sends individual JPEG frames as binary data over a Socket.IO WebSocket, and the frontend draws each frame onto an HTML5 `<canvas>` element.

This approach was chosen because the server needs to draw hand landmarks (colored dots and lines) on each frame *before* sending it — a plain `<video>` stream cannot be pre-processed server-side. The canvas scales the image to fill its container without letterboxing, using a "cover" strategy.

```
socket.on('frame', data => {
  const blob = new Blob([data], { type: 'image/jpeg' });
  imgFeed.src = URL.createObjectURL(blob);  // decode JPEG
  // imgFeed.onload draws to canvas when ready
});
```

2.4 Recognition Panel — What Each Element Does

Large Letter Display

Shows the currently detected ISL sign in a huge Syne font, glowing amber. A burst animation triggers when a new letter is confirmed — scale from 78% to 100% with a white flash.

Confidence Bar

Shows how sure the model is about its prediction (0–100%). Green above 70%, amber 40–70%, red below 40%. Low confidence = the hand sign is unclear or between two poses.

Stability Bar

Counts how many consecutive frames showed the same prediction (max 8). A letter is only committed at 8/8 — this prevents accidentally typing a letter during hand movement.

Current Word Display

Shows the letters being spelled in real time. Grows one letter at a time as each letter reaches full stability. Shows underscore when empty.

2.5 Word Suggestion System

After each new letter, the frontend receives an updated list of up to 3 word suggestions from the server. These are words from a 120-word ISL vocabulary that *start with* what the user has typed so far.

Clicking a suggestion pill (or pressing **1**, **2**, or **3** on the keyboard) sends an `act('suggest', index)` request to the server, which: commits the full suggested word to the sentence, clears the current word, and — critically — **resets the stability counter to zero**. Without that reset, the held hand sign would immediately append to the new empty word before the user has time to move their hand.

2.6 Text-to-Speech — Web Speech API

Speaking is handled entirely inside the browser using the **Web Speech API** (`window.speechSynthesis`) — a built-in browser feature that requires no extra software or Python packages. This works on Windows (Microsoft voices), Mac (Siri voices), and Linux (eSpeak or other installed voices).

The flow is: user clicks SPEAK → browser sends `act('enter')` to server → server emits a `'tts'` Socket.IO event with the sentence text → browser receives it and calls `speakText(text)` :

```
function speakText(text) {
  window.speechSynthesis.cancel();         // stop any current speech
  const utt = new SpeechSynthesisUtterance(text);
  utt.voice = selectedVoice;               // chosen from voice dropdown
  utt.rate  = ttsRate;                     // 0.5× to 2× speed
  utt.onstart = () => showSpeaking(true);  // show wave animation
  utt.onend   = () => showSpeaking(false);
  window.speechSynthesis.speak(utt);
}
```

> **IN SIMPLE TERMS**
>
> The browser already knows how to speak text — every modern browser has this built in. We just tell it "say these words in this voice at this speed." When the user picks a voice from the dropdown, they're choosing from the voices installed on their computer's operating system, not downloading anything new.

2.7 All Features and How to Use Them

Feature	Where to Find It	What It Does	Key Press
␣ SPACE	Recognition panel	Commits current word to sentence, clears current word	Spacebar
▶ SPEAK	Recognition panel	Speaks full sentence including any uncommitted word	Enter
⌫ BACK	Recognition panel	Deletes last letter, or last word if word is empty	Backspace
✕ CLEAR	Recognition panel	Clears everything — full sentence and current word	—
Suggestion pills	Below current word	Accept a word completion suggestion	1 / 2 / 3
AUTO-SPEAK	Header toggle	Automatically speaks each word as it is committed with Space	—
Voice selector	Header dropdown	Choose from any voice installed on the operating system	—
SPEED slider	Header	Adjust TTS speech rate (0.5× slow to 2× fast)	—
▶ SPEAK ALL	Sentence bar	Speaks the complete sentence from the sentence bar	Enter

Feature	Where to Find It	What It Does	Key Press
⎘ **COPY**	Sentence bar	Copies sentence text to clipboard, shows toast confirmation	—
▤ **HISTORY**	Sentence bar	Opens a panel showing the last 10 spoken sentences; click ▶ to re-speak any	—
ISL Reference	Right panel	Shows all 26 alphabet and 9 gesture cards; auto-highlights the current detected sign	—

2.8 Speaking Wave Animation

When speech is active, five vertical bars animate in a wave pattern in the sentence bar. This is built entirely with CSS — no JavaScript animation loop needed. Each bar has a different `animation-delay`, creating a natural ripple effect:

```
.wave-bar { animation: wave 1.1s ease-in-out infinite; }
.wave-bar:nth-child(2) { animation-delay: 0.15s; }
/* ... */
@keyframes wave {
  0%,100% { transform: scaleY(0.4); }
  50%      { transform: scaleY(1.0); }
}
```

PART 03 · MEMBER 3 · EQUAL WEIGHT

Backend & ML Pipeline

Flask server, Socket.IO real-time communication, MediaPipe hand detection, 132-feature extraction, and Random Forest classifier

◆ BACKEND & MACHINE LEARNING

3.1 What the Backend Does

The backend is the Python program (`isl_main.py`) running on the computer. It has three jobs: (1) capture video from the webcam and process it with AI to detect signs, (2) serve the web dashboard to the browser, and (3) handle all user actions (space, speak, record, train). Everything the frontend shows comes from the backend.

> **IN SIMPLE TERMS**
>
> If the frontend is the face of the application, the backend is the brain and muscles. The user never directly interacts with the backend — they click buttons on the frontend, which sends a message to the backend, which processes it and sends back the result. The backend also runs the camera and AI constantly in the background, even when the user isn't clicking anything.

3.2 Flask + Socket.IO — Server Architecture

Flask is a lightweight Python web framework that handles HTTP requests — it serves the HTML dashboard page and handles button presses (POST requests). **Flask-SocketIO** adds real-time two-way communication on top of Flask using WebSocket connections.

The key insight: instead of the browser asking "what's the current state?" every 100ms (polling), the server *pushes* state to the browser the moment it changes — this is how the letter display updates with zero delay. The server also pushes video frames as binary data, which the browser renders on canvas.

```
sio = SocketIO(app, async_mode="threading", cors_allowed_origins="*")

# Push state every frame (no request needed from browser):
sio.emit('state_update', { 'letter': 'A', 'confidence': 0.94, ... })
sio.emit('frame', jpeg_bytes)   # raw binary JPEG
```

3.3 MediaPipe — Detecting Hand Landmarks

MediaPipe is Google's open-source machine learning library for perception tasks. We use its **Hand Landmarker** model, which locates 21 specific points on each hand (fingertips, knuckles, wrist) in 3D space:

- Landmark 0: Wrist · Landmarks 1–4: Thumb · Landmarks 5–8: Index finger
- Landmarks 9–12: Middle · Landmarks 13–16: Ring · Landmarks 17–20: Pinky

Each landmark has an **x** (horizontal), **y** (vertical), and **z** (depth) coordinate, all normalized 0–1 relative to the image size. The model handles up to 2 hands simultaneously and also identifies which hand is left and which is right.

> **IN SIMPLE TERMS**
>
> Imagine pressing your hand flat on a piece of paper and marking 21 dots — one on your wrist, four along each finger. MediaPipe does this automatically in real time, finding all 21 dots on each hand in every frame, even as the hand moves. It outputs the (x, y, z) coordinate of each dot, which our code then uses to figure out what sign is being made.

3.4 Feature Extraction — The 132 Numbers

Raw landmark coordinates are useless directly — if you stand closer to the camera, all 42 coordinates scale up; if you move your hand left, all x values shift. The classifier must work regardless of distance, position, or hand size. Feature extraction solves this by making the coordinates **scale- and position-invariant**:

1. **Subtract the wrist** (landmark 0) from all 21 landmarks. Now all coordinates are relative to the wrist — moving your hand left/right/up/down changes nothing.
2. **Divide by a scale factor** — the distance between the wrist and the middle-finger base (landmark 9). Now all coordinates are proportional to hand size — distance from camera changes nothing.
3. **Flatten to a list:** 21 landmarks × 3 axes = 63 numbers per hand. Right hand = 63, left hand = 63. If a hand isn't detected, its 63 values are all zero.
4. **Add 6 relative features** between the two hands: Δx, Δy, Δz between wrists, wrist-to-wrist distance, index fingertip gap, thumb tip gap.

> **TOTAL: 63 + 63 + 6 = 132 FEATURES**
>
> This vector is a complete, normalized "fingerprint" of the hand pose that the Random Forest classifier uses to identify the ISL sign.

3.5 Random Forest Classifier — How It Works

A **Random Forest** is an ensemble of many decision trees (we use 100). Each decision tree learns rules like: "If feature 7 is greater than 0.3 AND feature 22 is less than -0.1, predict letter A." Each tree was trained on a slightly different random subset of the training data, making each tree slightly different. When classifying a new hand pose, all 100 trees vote, and the majority vote wins.

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(
    n_estimators = 100,   # 100 decision trees
    max_depth    = 15,    # max 15 levels deep per tree
    n_jobs       = -1     # use all CPU cores in parallel
)
clf.fit(X_train, y_train)   # learn from recorded samples
pred = clf.predict(feat)    # predict sign for new frame
conf = max(clf.predict_proba(feat)[0])  # confidence (0–1)
```

> **IN SIMPLE TERMS**
>
> A Random Forest is like asking 100 different experts the same question and going with the majority answer. Each expert learned from slightly different examples, so they disagree on edge cases — but when they agree, you can be confident. The confidence percentage shows how many of the 100 trees agreed: 94% confidence means 94 out of 100 trees voted for the same letter.

3.6 The Stability Threshold System

Even a good model produces noisy predictions during hand transitions (moving from one sign to the next). Without filtering, letters would flash rapidly during every hand movement. The stability system fixes this:

```
if prediction == last_prediction AND confidence > 50%:
    stable_count += 1  (max 8)
else:
    stable_count -= 1  (min 0)
    last_prediction = new_prediction

if stable_count >= 8 AND prediction != current_letter:
    commit letter to word  # only now does the letter appear
```

A letter must be the dominant prediction for **8 consecutive frames** (about 0.27 seconds at 30fps) before it is accepted. This eliminates false detections during transitions without adding noticeable delay for deliberate signs.

3.7 REST API Endpoints

Endpoint	Method	What It Does
`/action`	POST	Central command handler: space, enter (speak), backspace, clear, suggest, auto_speak, clear_history
`/start_recording`	POST	Begin saving feature vectors to `isl_dataset.csv` with the given label

Endpoint	Method	What It Does
`/stop_recording`	POST	Stop saving, return updated sample counts
`/train`	POST	Launches model training in a background thread (non-blocking)
`/dataset_info`	GET	Returns JSON: sample count for every class (A–Z + gestures)
`/delete_latest`	POST	Removes the last recorded row for a given class from the CSV
`/clear_class`	POST	Deletes all rows for a given class from the CSV

3.8 Socket.IO Events (Real-Time Pushes)

Event Name	Direction	Contains
`frame`	Server → Browser	Raw JPEG bytes (binary) — rendered to canvas each frame
`state_update`	Server → Browser	Detected letter, confidence, stability count, current word, sentence, suggestions, hand flags
`tts`	Server → Browser	`{text: "..."}` — triggers Web Speech API in browser
`history_update`	Server → Browser	`{history: [...]}` — updates the spoken sentence history panel
`training_status`	Server → Browser	`{status, accuracy, error}` — updates training progress display
`sample_recorded`	Server → Browser	`{label, count}` — live counter during data recording

PART 04 · MEMBER 4 · SHORTEST SECTION

Dataset Collection, Training & Deployment

How to record training data, train the model, use the full system, and run it on Windows — step by step

◆ DATA, TRAINING & HOW TO USE

4.1 What is Training Data and Why Do We Need It?

The machine learning model cannot recognize ISL signs on its own — it has to be **taught** by example. Training data is a collection of hand poses recorded by a real person making each sign, labeled with the correct sign name. The more examples the model sees of each sign, the better it gets at recognizing them.

> **IN SIMPLE TERMS**
>
> Teaching the model is like teaching a child to recognize letters. You show them the letter "A" many times from different angles, and eventually they recognize it even when someone else writes it slightly differently. We do the same — we record ourselves making the ISL sign for "A" hundreds of times, and the model learns to recognize that hand shape even when someone else makes it.

4.2 Step-by-Step — Collecting Training Data

Open the browser at **http://localhost:5000** and click the **TRAINER** tab at the top.

1. **Pick a sign to record:** Use the dropdown at the top to select which letter or word you want to record (e.g. select "A").
2. **Make the sign:** Hold your hand(s) in the correct ISL posture in front of the webcam. The live camera feed on the left helps you check your position.
3. **Hold the RECORD button:** Press and hold the red circular RECORD button (or hold the **R** key). The REC indicator flashes red while recording. The sample counter ticks up in real time.
4. **Release after 2–3 seconds:** Let go of the button. The samples are saved automatically. Repeat steps 2–4 a few more times with slight variations in angle.
5. **Switch to the next sign:** Choose a different letter in the dropdown and repeat. Aim for at least **100 samples per sign** for good accuracy.

> **◆ TIPS FOR BETTER DATA**
>
> Record in the same room and lighting you plan to use the system in. Try slightly different hand distances (30–60 cm from camera). If a recording looks wrong, click **DELETE LATEST** to remove the last batch, or **CLEAR ALL** to start that sign from scratch.

4.3 Training the Model

Once you have data for at least 2 signs (and at least 10 total samples), click the **TRAIN MODEL** button in the Trainer tab. The button shows a spinner while training runs in the background — this takes 5 to 30 seconds depending on dataset size. When done, the status shows *"Success — model loaded"* and the test accuracy percentage.

The model is immediately live — switch to the Recognition tab and test it right away. No restart needed.

4.4 How to Use the Recognition System

What You Want To Do	How To Do It
Sign a letter	Make the ISL hand shape and hold it steady. The letter appears when the stability bar fills to 8/8.
Finish a word	Press **Space** or click ␣ SPACE to commit the word to the sentence.

What You Want To Do	How To Do It
Accept a suggestion	After a few letters, word suggestions appear. Click the pill or press **1, 2, or 3**.
Speak the sentence	Click ▶ **SPEAK ALL** in the bottom bar, or press **Enter**.
Auto-speak mode	Turn on AUTO-SPEAK in the header — each word is spoken as soon as Space is pressed.
Correct a mistake	Press **Backspace** to delete the last letter, or click ⌫ BACK.
Start over	Click ✕ **CLEAR** to erase everything.

4.5 Running on Windows — Step by Step

1. **Copy the project folder** to the Windows PC. Make sure `hand_landmarker.task`, `isl_model.pkl`, and `labels.pkl` are all present.
2. **Install Python 3.10+** from python.org. During setup, check "Add Python to PATH."
3. **Double-click** `start.bat` in the project folder. It installs all packages and starts the server automatically.
4. A browser tab opens at **http://localhost:5000**. Windows TTS voices appear in the voice dropdown with no extra setup.

4.6 Troubleshooting

Problem	Fix
Camera not opening	Close any other app using the camera (Zoom, Teams). Try a different USB port.
Speak button is silent	Allow audio in the browser. Check system volume. Try a different voice in the dropdown.
Low recognition accuracy	Collect more samples (150+) per sign. Retrain the model.
Model mismatch warning	An old model is loaded. Collect new two-handed data in the Trainer tab and retrain.
Port 5000 in use	Run `taskkill /F /IM python.exe` (Windows) or `pkill python3` (Linux) then restart.